OS & SP

Name : M Ramya Sree **Roll No :** 2520030094

SKILL – 04

1.Skill4.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_INPUT 1024
typedef struct {
    char **tokens;
    int count;
} TokenList;
/* Free allocated token memory */
void free_tokens(TokenList *list)
{
    for (int i = 0; i < list->count; i++) {
        free(list->tokens[i]);
```

```
    }
    free(list->tokens);
    list->tokens = NULL;
    list->count = 0;
}
/* Add a token to the token list */
void add_token(TokenList *list, const char *start, int length)
{
    char *token = malloc(length + 1);
    if (token == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }
    strncpy(token, start, length);
    token[length] = '\0';
    char **temp = realloc(
        list->tokens,
        (list->count + 1) * sizeof(char *)
```

```
    );
    if (temp == NULL) {
        free(token);
        perror("realloc");
        exit(EXIT_FAILURE);
    }
    list->tokens = temp;
    list->tokens[list->count] = token;
    list->count++;
}
/* Tokenize input */
TokenList tokenize(const char *input)
{
    TokenList list;
    list.tokens = NULL;
    list.count = 0;
    int i = 0;
    while (input[i] != '\0') {
```

```
/* Ignore whitespace */
while (isspace((unsigned char)input[i])) {
    i++;
}
if (input[i] == '\0') {
    break;
}
int start = i;
/* Read until whitespace */
while (input[i] != '\0' &&
       !isspace((unsigned char)input[i])) {
    i++;
}
add_token(
    &list,
    input + start,
    i - start
);
```

```
    }
    return list;
}
/* Validate tokens */
int validate_tokens(const TokenList *list)
{
    if (list->count == 0) {
        printf("Validation: Empty command\n");
        return 0;
    }
    printf("Validation: Token stream is valid\n");
    return 1;
}
/* Display tokens */
void print_tokens(const TokenList *list)
{
    printf("\n--- Tokenization Result ---\n");
```

```
    for (int i = 0; i < list->count; i++) {
        printf("Token %d: [%s]\n",
               i + 1, list->tokens[i]);
    }
    printf("Total tokens: %d\n", list->count);
}
int main()
{
    char input[MAX_INPUT];
    printf("====================================\n");
    printf("       OSSP PROGRAM 4\n");
    printf("       Tokenization Parser\n");
    printf("====================================\n");
    printf("Enter a command.\n");
    printf("Type 'exit' to quit.\n\n");
    while (1) {
        printf("parser> ");
```

```
if (fgets(input, sizeof(input), stdin) == NULL) {
    break;
}
/* Remove newline */
input[strcspn(input, "\n")] = '\0';
/* Handle empty input */
if (strlen(input) == 0) {
    printf("Error: Empty input\n\n");
    continue;
}
/* Exit condition */
if (strcmp(input, "exit") == 0) {
    break;
}
/* Tokenize input */
TokenList list = tokenize(input);
/* Print tokens */
print_tokens(&list);
```

```
    /* Validate token stream */
    validate_tokens(&list);
    /* Free memory */
    free_tokens(&list);
    printf("\n");
  }
  printf("Parser terminated successfully.\n");
  return 0;
}
```

